Programmation et Algorithme

❖ Les Operateurs :

Les operateurs Arithmetiques

Python	Algorithme
+	+
-	-
*	*
/	/
//	div
%	mod

Les operateurs Logiques

Python	Algorithme
not	non
and	et
or	ou

Les operateurs de Comparaison

Python	Algorithme
==	=
!=	≠
>	>
>=	⩾
<	<
<=	⩽
a <= N <= b	a ⩽ N ⩽ b ou N € [a..b]
N in [a,b,c,d]	N € [a,b,c,d

❖ Les Fonctions et les Methodes prédéfinie:

Les Fonctions et les Methodes sur les entiers / reels

Python	Algorithme	Role
str(N)	convch(N)	retourne la convertion d'un entier N en une chaine de caracteres
chr(N)	chr(N)	retourne le caractère dont le code ASCII est N (entier)
abs(N)	abs(N)	retourne la valeur absolue d'un entier N
float(N)	valeur(N)	retourne la valeur reel d'un entier N
round(N)	arrondi(N)	retourne l'entier le plus proche de N
randint(vd,vf)	alea(vd,vf)	retourne un entier aleatoire de l'intervalle [vd,vf] nb : En Python, vous devez importer la bibliothèque random : from random import *
sqrt(N)	racine(N)	retourne la racine carree de N nb : En Python, vous devez importer la bibliothèque mathématique : from math import *
floor(N)	ent(N)	retourne l'entier la partie de N nb : En Python, vous devez importer la bibliothèque mathématique : from math import *

Les Fonctions et les Methodes sur les caracteres

Python	Algorithme	Role
ord(c)	ord(c)	retourne le code ASCII du caractere c
int(c) float(c)	valeur(c)	retourne la convertion d'une caractere c en convertible en une valeur numerique (entier ou reel), si c'est possible
ch.find(c)	pos(c,ch)	retourne la premiere position de le caractere c dans la chaine ch
c.isdigit()	EstNum(c)	retourne Vrai si le caractere c est convertible en une numerique, elle retourne Faux sinon
c.upper()	Majus(c)	convertir le caractere c en majuscule

Les Fonctions et les Methodes sur les chaine caracteres

Python	Algorithme	Role
int(ch) float(ch)	valeur (ch)	retourne la convertion d'une chaine ch en convertible en une valeur numerique (entier ou reel), si c'est possible
len(ch)	long(ch)	retourne le nombre des caracteres de la chaine ch
ch2.find(ch1)	pos(ch1,ch2)	retourne la premiere position de la chaine ch1 dans la chaine ch2
ch[d : f]	sous_chaine(ch,d,f)	retourne une partie de la chaine ch a partir de la position d jusqu'a la position f (f excule)
ch.isdigit()	EstNum(ch)	retourne Vrai si la chaine ch est convertible en une numerique, elle retourne Faux sinon
ch = ch[: d] + ch[f :]	efface(ch,d,f)	Effaces des caracteres de la chaine ch a partir de la position d jusqu'a la position f (f excule)
ch.upper()	Majus(ch)	convertir la chaine ch en majuscules

❖ Les Types :

- **Type Simple**

  Reel : 3.14159265359 5.0 1567.999

  Entier : -3 -2 -1 0 1 2 3 4 5 6 7 9 10

  Caractere : "a" "A" "C" "+" "0" "1" "5" ")" "__"(space)

  Chaine de caracteres : "Bonjour" "Programmation" "Algorithme"

  Booleen : Vrai Faux

- **Type Complex**

  - ***Tableau :***

T = array([type] * taille)

Exemple : **T = array([int] * 5)**

1	2	3	4	5
0	1	2	3	4

← La position des éléments du tableau

T[0] = 1 **T[1] = 2** **T[3] = 4** **T[4] = 5**

T.D.O	
objet	**type**
T	tab

T.D.N.T
type
tab : tableau de 5 entier

  - ***Matrice :***

M = array([[type] * nombre de colonnes] * nombre de lignes)

Exemple : **M = array([[int] * 5] * 5)**

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Lignes

Colonnes

M[0, 4] = 5 **M[1, 1] = 7** **M[3, 0] = 167** **M[5, 5] = 25**

T.D.O	
objet	**type**
M	mat

T.D.N.T
type
mat : matrice de 5 * 5 entier

- ***Enregistrement :***

Python : e = dict( ) ou e = { }

e["champ"] = valeur

Algorithme : e.champ ← valeur

Exemple :

e = dict( )

e["model"] = "Galaxy S23 Ultra"

e["prix"] = 1300

e.model ← "Galaxy S23 Ultra"

e.prix ← 1300

{"model" : "Galaxy S23 Ultra", "prix" : 1300}

T.D.O	
objet	**type**
e	smartphone

T.D.N.T
type
smartphone : Enregistrement model : chaine prix : entier Fin

- ***Fichier texte :***

1) L'ouverture :

Python : nom logique = open(nom physique, mode)

Algorithme : ouvrir(nom physique, nom logique, mode)

mode :

w : pour ouvrir et écrire dans le fichier

r : pour lire le fichier

a : pour ajouter au fichier

Exemple :

f = open("nombre.txt", "r")

ouvrir("nombre.txt", f, "r")

2) Lecture :

Python :

ch = f.readline( )

ch = ch.rstrip( ) ou ch = ch[: len(ch1) - 1]

Algorithme :

Lire-ligne(f, ch)

3) L'écriture:

Python : f.write(ch + "\n")

Algorithme : ecrire-nl(f, ch)

4) Fermeture

Python : f.close( )

Algorithme : fermer(f)

- ***Fichier binaire :***

1) L'ouverture :

Python : nom logique = open(nom physique, mode)

Algorithme : ouvrir(nom physique, nom logique, mode)

mode :

wb : pour ouvrir et écrire dans le fichier

rb : pour lire le fichier

ab : pour ajouter au fichier

Exemple :

```
f = open("voiture.dat", "wb")
ouvrir("voiture.dat", f, "wb")
```

2) Lecture :

Python : e = load(f)

Algorithme : lire(f, e)

3) L'écriture:

Python : dump(e, f)

Algorithme : ecrire(f, e)

4) Fermeture

Python : f.close( )

Algorithme : fermer(f)

T.D.O	
objet	**type**
e	voiture
f	fiche

T.D.N.T
type
voiture : Enregistrement model : chaine prix : entier Fin fiche = fichier de voiture

Remarque :

- Le type de contenu d'un fichier texte est chaine de caracteres
- Le type de contenu d'un fichier binaire est enregistrement
- Pour créer un tableau ou une matrice en python, vous devez ajouter la bibliothèque numpy en écrivant **from numpy import *** dans la première ligne du code
- Pour écrire ou lire un fichier binaire en python, vous devez ajouter la bibliothèque pickle en écrivant **from pickle import *** dans la première ligne du code
- Si vous utilisez la méthode pk.dump, vous devez ajouter la bibliothèque pickle via **import pickle as pk**

❖ **Les entrees :**

Python :

ch = input("donner un valeur") si ch est de type caractere ou chaine

n = int(input("donner un entier")) si n est de type entier

n = float(input("donner un reel")) si n est de type reel

Algorithme :

ecrire("donner un valeur")	si x est de type caracatere ou chaine
lire(x)	ou entier ou reel

❖ **Les sorties :**

Python : print("La somme est : ", S)

Algorithme : ecrire("La somme est : ", S)

❖ **La structure conditionnelle :**

Python	Algorithme
if Condition 1 : Traitement 1 elif Condition 2 : Traitement 2 . . . elif Condition N-1 : Traitement N-1 else : Traitement N	Si Condition 1 alors Traitement 1 Sinon Si Condition 2 alors Traitement 2 . . . Sinon Si Condition N-1 alors Traitement N-1 Sinon Traitement N finsi
Exemple : S = 7 if S == 1 : print("condition 1") elif S != 7 : print("condition 2") else : print("condition 3")	S ← 7 Si S = 1 alors ecrire("condition 1") Sinon Si S ≠ 7 alors ecrire("condition 2") Sinon ecrire("condition 3") finsi

❖ La structure iteratives (les boucles) :

Python	Algorithme	
for cpt in range(d, f + 1): Traitement	Pour cpt de d à f faire Traitement Finpour	Si d < f
for cpt in range(d, f - 1, -1): Traitement	Pour cpt de d à f (pas = -1) faire Traitement finpour	Si d > f
While condition : Traitement	Tantque condition faire Traitement fintantque	
test = False While not(test) : Traitement test = condition	Repeter Traitement Jusqu ' à condition	

- **For / Pour :**

1) d < f :

Python	Algorithme
for i in range(5): print(i)	Pour i de 0 à 4 faire ecrire(i) Finpour
>>> 0 1 2 3 4	

2) d > f :

Python	Algorithme
for i in range(5, 0, -1): print(i)	Pour i de 5 à 1 (pas = -1) faire ecrire(i) Finpour
>>> 5 4 3 2 1	

Remarque :

- En python, si d = 0, on peut l'omettre
- Dans l'algorithme, le début et la fin que vous spécifiez sont affichés
- Contrairement à python, vous devez en ajouter 1 à la fin

- **While / Tantque :**

Python	Algorithme
i = 1 while i < 6 : print(i) i = i + 1	i ← 1 Tantque i < 6 faire ecrire(i) i ← i + 1 fintantque
>>> 1 2 3 4 5	

- **While / Repeter :**

Python	Algorithme
i = 1 test = False While not(test) : print(chr(i + 64)) i = i + 1 test = (i >= 6)	i ← 1 Repeter ecrire(chr(i + 64)) i ← i + 1 Jusqu ' à (i >= 6)
>>> A B C D E	

❖ Les sous programmes :

- Les sous-programmes, également appelés fonctions ou procédures, sont des blocs de code réutilisables et autonomes qui effectuent une tâche spécifique dans un programme informatique. Les sous-programmes peuvent être appelés plusieurs fois à partir de différents endroits dans un programme, ce qui les rend très utiles pour réduire la redondance de code et pour faciliter la maintenance et la compréhension du code.
- Les fonctions sont des sous-programmes qui retournent une valeur calculée à partir de paramètres passés en entrée. Les procédures, quant à elles, sont des sous-programmes qui ne retournent pas de valeur mais qui peuvent modifier les variables ou l'état du programme.
- En général, les sous-programmes permettent de simplifier la conception et la mise en œuvre de programmes complexes en permettant la modularité, la réutilisation et la structuration du code. Ils peuvent également améliorer la lisibilité du code et permettre une maintenance plus facile en permettant de modifier ou de corriger un bloc de code spécifique sans affecter le reste du programme.

- **La fonction :**
  - **Forme générale :**

Python :

```
def nomfonction(parametre formels) :
    Traitement
    return valeur
```

Algorithme :

```
fonction nomfonction(parametre formels:type):type de valeur retour
Debut
    Traitement
    retourner valeur
Fin
```

  - **l'appel de la fonction :**

nomfonction(parametre effectifs)

- **en variables :**

Python : x = nomfonction(parametre effectifs)

Algorithme : x ← nomfonction(parametre effectifs)

  - **Exemple :**

Python :

```
def puiss(x, y):
    p = 1
    for i in range(y):
        p = p * x
    return p
s = puiss(5, 2)
print(s)
>>> 25
print(puiss(4, 2))
>>> 16
```

parametre formels : x, y

parametre effectifs : 5, 2

Algorithme :

```
fonction puiss(x, y : entier): entier
Debut
    p ← 1
    pour i de 1 à y faire:
        p ← p * x
    finpour
    retourner p
Fin
s ← puiss(5, 2)
ecrire(s)
>>> 25
ecrire(puiss(4, 2))
>>> 16
```

- **La Procedure :**
  - **Forme générale :**

Python :

```
def nomfonction(parametre formels) :
    Traitement
```

Algorithme :

```
procedure nomfonction(parametre formels:type)
Debut
    Traitement
Fin
```

- **l'appel de la fonction :**

nomfonction(parametre effectifs)

Remarque :

- La procédure ne renvoie aucune valeur, elle met à jour et modifie des variables ou l'état du programme principal
- La procédure faire la remplir le tableau, la matrice, ou le fichier, ou les modifie, elle fait aussi un affichage
- Nous allons utiliser la procédure dans quelques exemples

❖ Les entier et reels :

Python	Algorithme
ch = str(16) print(ch)	ch ← convch(16) ecrire(ch)
>>> 16 (type chaine)	
c = chr(65) print(c)	c ← chr(65) ecrire(c)
>>> A (type caractere)	
n = abs(2 - 7) print(n)	n ← abs(2 - 7) ecrire(n)
>>> 5 (type entier)	
x = float(7) print(x)	x ← valeur(7) ecrire(x)
>>> 7.0 (type reel)	
x = round(5.76545, 2) print(x)	x ← arrondi(5.76545, 2) ecrire(x)
>>> 5.77 (type reel)	
from random import * n = randint(0, 50) print(n)	n ← alea(0, 50) ecrire(n)
>>> 14 (type entier)	
from math import * x = sqrt(25) print(x)	x ← racine(25) ecrire(x)
>>> 5.0 (type reel)	
from math import * x = floor(100.999) print(x)	x ← ent(100.999) ecrire(x)
>>> 100 (type entier)	

❖ Les Caracteres :

Python	Algorithme
n = ord("C") print(n)	n ← ord("C") ecrire(n)
>>> 67 (type entier)	
n = int("7") print(n)	n ← valeur("7") ecrire(n)
>>> 7 (type entier)	
x = float("5") print(x)	x ← valeur("5") ecrire(x)
>>> 5.0 (type reel)	
ch = "ABCDEF" n = ch.find("C") print(n)	ch ← "ABCDEF" n ← pos("C", ch) ecrire(n)
>>> 2 (type entier)	
t = isdigit("1") print(t)	t ← EstNum("1") ecrire(t)
>>> True (type booleen)	
x = "b" c = x.upper( ) print(c)	x ← "b" c ← Majus(x) ecrire(c)
>>> B (type caractere)	

❖ Les Chaine de caracteres :

Python	Algorithme
n = int("37") print(n)	n ← valeur("37") ecrire(n)
>>> 37 (type entier)	
x = float("25.50") print(x)	x ← valeur("25.50") ecrire(x)
>>> 25.50 (type reel)	
n = len("hello world") print(n)	n ← long("hello world") ecrire(n)
>>> 11 (type entier)	
ch = "hello world" n = ch.find("world") print(n)	ch ← "hello world" n ← pos("world", ch) ecrire(n)
>>> 6 (type entier)	
ch1 = "Programmation" ch2 = ch1[0 : 7] print(ch2)	ch1 ← "Programmation" ch2 ← sous-chaine(ch1,0 , 7) ecrire(ch2)
>>> Program (type chaine)	
t = isdigit("123a") print(t)	t ← EstNum("123a") ecrire(t)
>>> False (type booleen)	
ch1 = "Python React Javascript" ch2 = ch1[: 6] + ch1[12 :] print(ch2)	ch1 ← "Python React Javascript" ch2 ← efface(ch1, 6, 12) ecrire(ch2)
>>> Python Javascript (type chaine)	
ch1 = "hello world" ch2 = ch1.upper( ) print(ch2)	ch1 ← "hello world" ch2 ← Majus(ch1) ecrire(ch2)
>>> HELLO WORLD (type chaine)	

- **Ajouter à la chaine :**

Python	Algorithme
ch = "" for i in range(5): ch = ch + str(i + 1) print(ch)	ch ← "" pour i de 0 à 4 faire ch ← ch + convch(i + 1) ecrire(ch)
>>> 12345 (type chaine)	
ch = "" for i in range(5): ch = str(i + 1) + ch print(ch)	ch ← "" pour i de 0 à 4 faire ch ← convch(i + 1) + ch ecrire(ch)
>>> 54321 (type chaine)	

❖ Les Tableau :

○ Remplir Tableau :

Python :

```
1  from numpy import *
2
3
4  def RemplirTab(n):
5      for i in range(n):
6          T[i] = i
7
8
9  T = array([int] * 5)
10 RemplirTab(5)
11 print(T)
12
```

Algorithme :

```
1  Procedure RemplirTab(@T : tab)
2  Debut
3      pour i de 0 à 4 faire
4          T[i] <-- i
5      finpour
6  Fin
7
```

○ Tri Tableau :

Python :

1) Tri bulle :

```python
def tri_bulle(n) :
    Trier = True
    while Trier :
        Trier = False
        for i in range(n - 1):
            if T[i] > T[i + 1] :
                aux = T[i]
                T[i] = T[i + 1]
                T[i + 1] = aux
                Trier = True

T = [14, 5, 20, 6, 16, 7]
tri_bulle(6)
print(T)
```

2) Tri insertion :

```python
def tri_insertion(n) :
    for i in range(1, n):
        j = i
        aux = T[i]
        while T[j - 1] > aux and j > 0 :
            T[j] = T[j - 1]
            j = j - 1
        T[j] = aux

T = [14, 5, 20, 6, 16, 7]
tri_insertion(6)
print(T)
```

3) Tri selection :

```python
def tri_selection(n) :
    for i in range(0, n - 1):
        pmin = i
        for j in range(i + 1, n):
            if T[j] < T[pmin] :
                pmin = j
        aux = T[i]
        T[i] = T[pmin]
        T[pmin] = aux
    return T

T = [14, 5, 20, 6, 16, 7]
tri_selection(6)
print(T)
```

4) Tri shell :

```python
def tri_shell(n) :
    pas = 1
    while pas < n :
        pas = 3 * pas + 1
    while pas != 1 :
        pas = pas // 3
        for i in range(pas, n):
            aux = T[i]
            j = i
            while T[j - pas] > aux and j >= pas :
                T[j] = T[j - pas]
                j = j - pas
            T[j] = aux

T = [7, 43, 16, 20, 14, 6, 5]
tri_shell(7)
print(T)
```

Algorithme :

1) Tri bulle :

```
Procedure Tri_Bulle(@T: tab, N: entier)
Debut
    Repeter
        trier <-- Vrai
        pour i de 0 à N - 2 faire
            Si T[i] > T[i + 1] alors
                aux <-- T[i]
                T[i + 1] <-- aux
                trier <-- Faux
            finsi
        finpour
    Jusqu'à ( trier = Vrai)
Fin
```

2) Tri insertion :

```
Procedure Tri_insertion(@T : tab, N: entier)
Debut
    pour i de 1 à N - 1 faire
        j <-- i
        aux <-- T[i]
        Tantque T[j-1] > aux et j > 0 faire
            T[j] <-- T[j - 1]
            j <-- j - 1
        finTantque
        T[j] <-- aux
    finpour
Fin
```

3) Tri selection :

```
Procedure Tri_Selection(@T : tab, N : entier)
Debut
    pour i de 1 à N - 2 faire
        min <-- i
        pour j de i + 1 à N - 2 faire
            Si T[j] < T[min] alors
                min <-- j
            finsi
        finpour
        aux <-- T[i]
        T[i] <-- T[min]
        T[min] <-- aux
    finpour
Fin
```

4) Tri shell :

```
Procedure Tri_Shell(@T : tab, N : entier)
Debut
    pas <-- 1
    Tantque pas < n faire
        pas <-- 3 * pas + 1
    finTantque
    Tantque pas ≠ 1 faire
        pas <-- pas div 3
        pour i de pas à N - 1 faire
            aux <-- T[i]
            j <-- i
            Tantque T[j - pas] > aux et j >= pas faire
                T[j] <-- T[j - pas]
                j <-- j - pas
            finTantque
            T[j] <-- aux
        finpour
    finTantque
Fin
```

○ Recherche dans tableau :

Python :

1) Recherche sequentielle :

```python
def RechercheSeq(tab, n, x):
    trouve = False
    i = 0
    while not trouve and  i < n :
        if tab[i] == x :
            trouve = True
        i = i + 1
    return trouve


T = [3, 5, 6, 7, 16, 20, 21, 34, 37, 55, 77, 144, 233, 377]
N = int(input("donner la valeur a cherche : "))
print(RechercheSeq(T, 14, N))
```

2) Recherche dichotomique :

```python
def RechercheDicho(tab, n, x):
    trouve = False
    Deb = 0
    Fin = n
    while not trouve and  Deb < Fin :
        mid = (Deb + Fin) // 2
        if tab[mid] > x :
            Fin = mid
        elif tab[mid] < x :
            Deb = mid + 1
        else :
            trouve = True
    return trouve


T = [3, 5, 6, 7, 16, 20, 21, 34, 37, 55, 77, 144, 233, 377]
N = int(input("donner la valeur a cherche : "))
print(RechercheDicho(T, 14, N))

```

Algorithme :

1) Recherche sequentielle :

```
Fonction RechercheSeq(T: tab, n,x: entier): booleen
Debut
    trouve <-- Faux
    i <-- 0
    Tantque trouve = Faux and i < n faire
        Si T[i] = x alors
            trouve = Vrai
        finsi
        i <-- i + 1
    finTantque
    retourner trouve
Fin
```

2) Recherche dichotomique :

```
Fonction RechercheDicho(T: tab, n,x: entier): booleen
Debut
    trouve <-- Faux
    D <-- 0
    F <-- n
    Tantque trouve = Faux and D < F faire
        m <-- (D + F) div 2
        Si T[m] > x alors
            F <-- m
        Sinon Si T[m] < x alors
            D <-- D + 1
        Sinon
            trouve <-- Vrai
        finsi
    finTantque
    retourner trouve
Fin
```

○ Remplir tableau from fichier:

Python :

1) from fichier text :

```python
from numpy import *

def taille( ):
    f = open("nombre.txt", "r")
    ch = f.readline( )
    nb = 0
    while ch != "" :
        nb = nb + 1
        ch = f.readline( )
    return nb

def RemplirTab(n):
    f = open("nombre.txt", "r")
    for i in range(n):
        ch = f.readline( )
        ch = ch.rstrip( )
        T[i] = ch
    f.close( )

T = array([str] * taille( ))
RemplirTab(taille( ))
print(T)
```

2) from fichier binaire :

```python
from numpy import *
from pickle import *

def taille( ):
    f = open("nombre.dat", "rb")
    fin = False
    nb = 0
    while not fin :
        try :
            load(f)
            nb = nb + 1
        except EOFError :
            fin = True
    f.close()
    return nb

def RemplirTab(n):
    f = open("nombre.dat", "rb")
    for i in range(n):
        en = load(f)
        T[i] = en
    f.close( )

T = array([dict( )] * taille( ))
RemplirTab(taille( ))
print(T)
```

Algorithme :

1) from fichier Text / Binaire :

```
Procedure RemplirTab(@T: tab, ch1 : chaine, n: entier)
Debut
    ouvrir(ch1, f1, "r")          /* rb
    pour i de 0 à n - 1 faire
        Lire-ligne(f, ch)         /* Lire(f, e)
        T[i] <-- ch               /* T[i] <-- e
    finpour
    fermer(f1)
Fin

Fonction taille(ch1 : chaine): entier
Debut
    ouvrir(ch1, f1, "r")
    nb <-- 0
    Tantque non(fin-fichier(f1)) faire
        Lire-ligne(f, ch)
        nb <-- nb + 1
    finTantque
    fermer(f1)
    retourner nb
Fin

##### Algorithme PP #####
Debut
ch1 <-- "nombre.txt"
N <-- taille(ch1)
RemplirTab(T, ch1, N)
Fin
#########################
```

* **Dans l'algorithme, la plupart du temps, vous n'êtes pas obligé de connaître la longueur du tableau, et il vous indique N est déjà saisi dans le programme principal, donc c'est comme suit :**

```
Procedure RemplirTab(@T : tab, ch1: chaine)
Debut
    ouvrir(ch1, f1, "r")                    /* rb
    i <-- 0
    Tantque non(fin-fichier(f1)) faire
        Lire-ligne(f, ch)                   /* Lire(f, e)
        T[i] <-- ch                         /* T[i] <-- e
        i <-- i + 1
    finTantque
    fermer(f1)
Fin
```

❖ Matrice :

○ Remplir Matrice :

Python :

```python
from numpy import *


def RemplirMat(C, L):
    for i in range(L):
        for j in range(C):
            # n = int(input("donner un valeur de M[" + str(i) + "," + str(j) + "] : ")) == Lire(M[i, j)
            M[i, j] = (i + 1) * (j + 1) # n


M = array([[int] * 5 ] * 5)
RemplirMat(5, 5)
print(M)

```

Algorithme :

```
Procedure RemplirMat(M: mat, L : entier, C: entier)
Debut
    pour i de 0 à L - 1 faire
        pour j de 0 à C - 1 faire
            Lire(M[i, j)
        finpour
    finpour
Fin


```

Remarque :

- S'il me reste du temps, je travaillerai sur un autre document pour remplir la matrice sous différentes formes

❖ Fichier Text :

○ Remplir Fichier Text :

* **Tout ce dont vous avez besoin pour remplir fichier text est de mettre**
  f.write(str(valeur) + "\n") ou ecrire-nl(convch(valeur))
  en boucle (for/pour) si vous savez quand vous arrêter
  ou en boucle (while/tantque si vous ne savez pas quand vous arrêter

```python
def RemplirF(deb, fin):
  f = open("nombre.txt", "w")
  for i in range(deb, fin):
    f.write(str(i) + "\n")
  f.close()

RemplirF(30, 51)
```

```
Procedure RemplirF(ch1: chaine, d,f: entier)
Debut
    ouvrir(ch1, f1, "w")
    pour i de d à f faire
        ecrire-nl(f1, convch(i))
    finpour
    fermer(f1)
Fin

##### Algorithme PP #####
Debut
NomF <-- "nombre.txt"
RemplirF(NomF, 30 , 51)
Fin
#########################
```

- **Remplir Fichier Text from tableau :**

  * **Comme ci-dessus, remplacez simplement i par T[i]**

  * **Mais si le tableau est rempli d'un enregistrement, il faut mettre le champ dans la variable ch et remplacer le i par ch**

    Vous en avez besoin dans la méthode de remplir from fichier binaire

- **Remplir Fichier Text from fichier binaire :**

Methode 1)

- **Remplir tableau from fichier binaire (page 16)**
- **Remplir le fichier from cet tableau (Comme ci-dessus)**

*** Vous avez besoin de cette méthode si vous voulez tri le fichier en faisant un tri pour le tableau (page 14/15)**

Methode 2)

- **Remplir directement from fichier binaire :**

```python
from pickle import *


def RemplirF( ):
  f1 = open("nombre.dat", "rb")
  f2 = open("nombre.txt", "w")
  fin = False
  while not fin :
    try :
      en = load(f1)
      ch = "NB 1 : " + str(en["nb1"]) + " " + "NB 2 : " + str(en["nb2"])
      f2.write(ch + "\n")
    except EOFError :
      fin = True
  f1.close( )
  f2.close( )

RemplirF( )

```

```
Procedure RemplirF(ch1,ch2: chaine)
Debut
    ouvrir(ch1, f1, "rb")
    ouvrir(ch2, f2, "w")
    Tantque non(fin-fichier(f1)) faire
        Lire(f1, en)
        ch <-- "NB 1 : " + convch(en.nb1) + " " + "NB 2 : " + convch(en.nb2)
        ecrire-nl(f2, convch(i))
    finpour
    fermer(f1)
    fermer(f2)
Fin

##### Algorithme PP #####
Debut
NF1 <-- "nombre.dat"
NF2 <-- "nombre.txt"
RemplirF(NF1, NF2)
Fin
#########################
```

❖ Fichier Binaire :

○ Remplir Fichier Binaire :

```python
from numpy import *
from pickle import *

|
def RemplirF(n):
   f = open("nombre.dat", "wb")
   en = dict( )
   for i in range(n):
      en["nb1"] = i
      en["nb2"] = i + 1
      dump(en, f)
   f.close( )

# From Tableau
def RemplirF(n):
   f = open("nombre.dat", "wb")
   for i in range(n):
      dump(T[i], f)
   f.close( )

T = array([dict( )] * 5) # ou array([{ }] * 5)
RemplirF(5)
```

```
Procedure RemplirF(ch1: chaine, n: entier)
Debut
    ouvrir(ch1, f1, "wb")
    pour i de 0 à n - 1 faire
        en.nb1 <-- i
        en.nb2 <-- i + 1
        Ecrire(f, en)
    finpour
    fermer(f1)
Fin

##### Algorithme PP #####
Debut
NF1 <-- "nombre.dat"
RemplirF(NF1, 5)
Fin
#########################
```

○ Tri Fichier Binaire :

- **Remplir Tableau from le fichier**
- **Trier le tableau**
- **Remplir le fichier from le tableau**

```python
def Tri_insertion(T, n):
   for i in range(1, n) :
      j = i
      aux = T[i]
      while T[j - 1]["NomChamp"] < aux["NomChamp"] and j > 0 :
         T[j] = T[j - 1]
         j = j - 1
      T[j] = aux
```

```
1   Procedure Tri_insertion(@T : tab, N: entier)
2   Debut
3       pour i de 1 à N - 1 faire
4           j <-- i
5           aux <-- T[i]
6           Tantque T[j-1].NomChamp > aux.NomChamp et j > 0 faire
7               T[j] <-- T[j - 1]
8               j <-- j - 1
9           finTantque
10          T[j] <-- aux
11      finpour
12  Fin
```

-

❖ Les Suite :

Soit une suite definie par

$$\begin{cases} U_0 = 2 \; U_1 = 3 \\ U_n = U_{n-1} + k * U_{n-2} \end{cases}$$

1) Calcul de la suite :

```
1   Fonction Suite(n,k:entier):entier
2   Debut
3       Si n = 0 alors
4           retourner 2
5       Sinon Si n = 1
6           retourner 3
7       Sinon
8           retourner (Suite(n-1) + k * Suite(n-2))
9       finsi
10  Fin
```

2) Verifier si x est un terme de la suite :

```
1   Fonction Verif(x: entier): booleen
2   Debut
3       n <-- 0
4       Tantque suite(n) < x faire
5           n <-- n + 1
6       finTantque
7       si suite(n) = x alors
8           retourner Vrai
9       Sinon
10          retourner Faux
11      finsi
12  Fin
```

Trianglaire Pascal :

```
1   Procedure Remplir(@M: Mat, N: entier)
2   Debut
3       pour i de 0 à N - 1 faire
4           pour j de 0 à N - 1 faire
5               si i < j ou i = j alors
6                   M[i,j] <-- 1
7               sinon
8                   M[i,j] <-- M[i - 1 , j] + M[i - 1 , j - 1 ]
9               finsi
10          finpour
11      finpour
12  Fin
```

Fibo Pascal :

```
1   Fonction Fibo_Pascal(n: entier): entier
2   Debut
3       s <-- 0
4       i <-- n
5       j <-- 0
6       Repeter
7           s <-- s + M[i,j]
8           i <-- i - 1
9           j <-- j + 1
10      Jusqua'à (i + j = N)
11  Fin
```

Suite Thue_Morse :

```
Fonction Thue_Morse (n: entier, U0: chaine): chaine
Debut
    pour i de 0 à n faire
        U0 <-- change(U0)
    finpour
    retourner U0
Fin

Fonction change(ch: chaine): chaine
Debut
    R <-- ""
    pour i de 0 à long(ch) - 1 faire
        Si ch[i] = "0" alors
            R <-- R + "01"
        Sinon
            R <-- R + "10"
        finsi
    finpour
    retourner R
Fin
```

Triangulaire / Triangulaire carre :

```
Fonction triangulaire(nb: entier): entier
Debut
    n <-- 1
    Repeter
        n <-- n + 1
    Jusqu'à (n * (n + 1) div 2 ≥ nbr)
    Si n * (n + 1) div 2 ≠ nbr alors
        n <-- 0
    finsi
    retourner n
Fin

Fonction triangulaire_carre(nb: entier): entier
Debut
    m <-- 1
    Repeter
        m <-- m + 1
    Jusqu'à (m * m ≥ nbr)
    Si m * m ≠ nbr ou triangulaire(m) = 0 alors
        m <-- 0
    finsi
    retourner m
Fin
```

Nombre D'or :

```
Fonction U(n: entier): entier
Debut
    si n = 1 ou n = 2 alors
        retourner 1
    sinon
        retourner (U(n - 1) + U(n - 2))
    finsi
Fin

Fonction V(n: entier): reel
Debut
    retourner (U(n) / U(n - 1))
Fin

Fonction Nbr_or(eps: reel): reel
Debut
    n <-- 2
    Repeter
        n <-- n + 1
    Jusqua'à (abs(V(n) - V(n-1)) < eps)
    retourner V(n)
Fin

```

❖ L 'Arithmetique :

1) PGCD :

```python
def pgcd(a, b):
  while a != b :
    if a > b :
      a = a - b
    else :
      b = b - a
  return a


def pgcd(a, b):
  while b != 0 :
    r = a % b
    a = b
    b = r
  return a

```

```
Fonction pgcd(a,b:entier):entier
Debut
    Tantque b ≠ 0 faire
        r <-- a mod b
        a <-- b
        b <-- r
    finTantque
    retourner a
Fin

# ------------------------------------------------

Fonction pgcd(a:entier, b:entier):entier
Debut
    Tantque a ≠ b faire
        Si a > b alors
            a <-- a - b
        sinon
            b <-- b - a
        finsi
    finTantque
    retourner a
Fin
```

2) PPCM :

```python
def ppcm(a, b):
  c = a
  d = b
  while a != b :
    if a > b :
      b = b + d
    else :
      a = a + c
  return a

```

```
Fonction ppcm(a,b:entier):entier
Debut
    c <-- a
    d <-- b
    Tantque a ≠ b faire
        Si a > b alors
            b <-- b + d
        Sinon
            a <-- a + c
        finsi
    finTantque
    retourner a
Fin

```

3) Somme Chiffre :

```python
def SommeChiffre(N):
  s = 0
  while N > 0 :
    R = N % 10
    N = N // 10
    s = s + R
  return s
```

```
Fonction Somme_Chiffres(n:entier):entier
Debut
    s <-- 0
    Repeter
        s <-- s + n mod 10
        n <-- n div 10
    Jusqu'à (n = 0)
    retourner s
Fin
```

4) Factoriel :

```python
def factorial(n):
  f = 1
  for i in range(1, n+1) :
    f = f * i
  return f
```

```
Fonction factorial(n:entier):entier
Debut
    f <-- 1
    pour i de 1 à n faire
        f <-- f * i
    finpour
    retourner f
Fin
```

5) Arrangement A^p_n :

```python
def arrangement(n, p):
  return fact(n) // fact(n - p)


def arrangement(n, p):
  d = n - p + 1
  a = 1
  for i in range(d, n + 1) :
    a = a * i
  return a
```

```
Fonction arrangement(n,p:entier):entier
Debut
    retourner fact(n) div fact(n - p)
Fin


Fonction arrangement(n,p:entier):entier
Debut
    d <-- (n - p) + 1
    a <-- 1
    pour i de d à n faire
        a <-- a * i
    finpour
    retourner a
Fin
```

6) Combinaison A^p_n :

```python
def factorial(x):
  f = 1
  for i in range(1, x+1) :
    f = f * i
  return f


def combinaison(n, p):
  return factorial(n) / (factorial(p) * factorial(n - p))

```

```
Fonction factorial(n:entier):entier
Debut
    f <-- 1
    pour i de 1 à n faire
        f <-- f * i
    finpour
    retourner f
Fin

Fonction combinaison(n,p:entier):reel
Debut
    retourner factorial(n) / (factorial(p) * factorial(n - p))
Fin

```

7) Premier :

```python
def premier(n):
  ok = True
  if n <= 1 :
    ok = False
  else :
    for i in range(2, n // 2) :
      if n % i == 0 :
        ok = False
  return ok

```

```
Fonction Premier(n:entier):booleen
Debut
    ok <-- Vrai
    Si n < 2 alors
        ok <-- Faux
    sinon
        pour i de 2 à (n div 2) faire
            Si n mod i = 0 alors
                ok <-- Faux
            finsi
        finpour
    finsi
    retourner ok
Fin
```

8) Puissance :

```python
def puissance(N, P):
  s = 1
  for i in range(P):
    s = s * N
  return s


```

```
Fonction puissance(n,p:entier):entier
Debut
    s <-- 1
    pour i de 1 à p faire
        s <-- s * n
    finpour
    retourner s
Fin

```

9) Decomposition en facteurs primier :

```python
def decomposition_facteurs_premiers(N):
  ch = ""
  d = 2
  while N > 1:
    if N % d == 0 :
      ch = ch + str(d) + "*"
      N = N // d
    else :
      d = d + 1
  return ch[ : len(ch) - 1]
```

```
Fonction DFacteurPremier(n:entier):chaine
Debut
    ch <-- ""
    d <-- 2
    Repeter
        Si n mod 2 = 0 alors
            ch <-- ch + convch(d) + "*"
            n <-- n div d
        Sinon
            d <-- d + 1
        finsi
    Jusqu'à (n = 1)
    retourner sous_chaine(ch, 0, long(ch)-1)
Fin
```

❖ Convertion Base :

1) Convertion 10 => Base :

```python
def Conv10_Base(N, B):
  alpha = "0123456789ABCDEF"
  ch = ""
  while N > 0 :
    R = N % B
    N = N // B
    ch = alpha[R] + ch
  return ch


```

```
Fonction Conv10_Base(N,B : entier) chaine
Debut
    alpha <-- "0123456789ABCDEF"
    ch <-- ""
    Tantque N > 0 faire
        R <-- N mod B
        N <-- N div B
        ch <-- alpha[R] + ch
    retourner ch
Fin


```

2) Convertion Base => 10 :

```python
def ConvBase_10(ch, b):
  alpha = "0123456789ABCDEF"
  s = 0
  p = 1
  for i in range(len(ch) - 1, -1, -1):
    s = s + alpha.find(ch[i]) * p
    p = p * b
  return s


```

```
Fonction ConvBase_10(ch: chaine, b: entier): entier
Debut
    alpha <-- "0123456789ABCDEF"
    s <-- 0
    p <-- 1
    pour i de long(ch) - 1 à 0 (pas = -1) faire
        s <-- s + pos(ch[i], alpha) * p
        p <-- p * b
    retourner s
Fin

```

❖ Approximation :

2) Valeur Approche :

```
Fonction pi(eps: reel): reel
Debut
    s <-- 0
    n <-- 1
    Repeter
        S2 <-- S2
        S <-- S + 6 / (n * n)
        n <-- n + 1
    Jusqu'à(abs(racine(S) - racine(S2)) > eps)
    retourner racine(S)
Fin

```

2) Calcul D'aire :

Methode Rectangle :

```
Fonction Rectangele_Melieu(a,b: reel, n: entier): reel
Debut           /* _Droite
                /* _Gauche

    h <-- (b - a) / n
    S <-- 0
    x <-- (a + h) / 2 /* Melieu
      /*(a + h)      /* Droite
      /*(a)          /* Gauche
    pour i de 1 à n faire
        S <-- S + h
        x <-- x + h
    finpour
    retourner S * f(x)
Fin

```

Methode Trapeze :

```
Fonction Trapeze(a,b: reel, n: entier): reel
Debut
    h <-- (b - a) / n
    S <-- 0
    x <-- a
    pour i de 1 à n faire
        S <-- S + f(x) + f(x + h)
        x <-- x + a
    finpour
    retourner S * h / 2
Fin

```

3) Optimisation :

```
Fonction Trapeze(eps: reel): reel
Debut
    x <-- 0
    Smax <-- 0
    Repeter
        x <-- x + eps
        S <-- 6 * x * 9 * (x * x) /* Donné
        Si S > Smax alors
            Smax <-- S
            Xopt <-- x
        finsi
    Jusqu'à x > 2 / 3 /* Donné
    retourner Xopt
Fin
```

4) Point fixe :

```
Fonction Pointfixe(eps: reel): reel
Debut
    x <-- 0
    Repeter
        x <-- f(x) /* Donné
    Jusqu'à (abs(f(x) - x) > eps)
    retourner x
Fin


```